

Trade-offs in Partition Assignment Policies intended for Distributed Deployment of Dynamic Graph Embedding Systems

Stefan Nožinić

Abstract

This paper studies trade-offs in partition assignment policies where partitioning balance and dynamic embedding quality are considered as two competing objectives. We describe a buffered event-processing pipeline in which incoming graph events are batched before a neighbor-based partitioner, with a tunable replication factor and capacity penalty, assigns vertices to partitions; embeddings are then maintained using a buffered variant of dynnode2vec. Using five real-world dynamic graph datasets (CITESEER, DBLP, AstroPh, AS-Oregon, Enron), we empirically evaluate how the number of partitions and the replication factor affect embedding quality, partition balance, edge cut, and repartitioning stability over time. Results show that the quality loss caused by partitioning is strongly dataset-dependent: on three of the five datasets (CITESEER, DBLP, Enron), embedding quality is stable or even improves as the partition count grows, while on the remaining two (AstroPh, AS-Oregon) it degrades. Among the dataset properties considered (density, average clustering coefficient, modularity), graph density correlates most closely with this split, though no single property fully explains it. We further show that increasing the replication factor can substantially recover the embedding quality lost to early, uncertain partition assignment, at the cost of additional computation, directly illustrating the need for flexibility in partitioning strategies.

Introduction

A graph is a mathematical structure consisting of vertices (or nodes) connected by edges. Graphs are widely used to model relationships and interactions in various domains [van_der_hofstad_random_2024], such as social networks [leskovec_signed_2010] [backstrom_group_2006] [rozemberczki_twitch_2021], collaboration networks [savic_analysis_2017], terrorist networks [krebs_mapping_2002] and blog citation networks [adamic_political_2005]. In these applications, the relationships between entities can be represented as edges connecting the corresponding vertices.

In many real-world graphs, which model real-world phenomena, there

are several properties that are often observed [watts_collective_1998] [zachary_information_1977] [albert_statistical_2002]: - **Small-world property:** Most pairs of vertices can be connected by a short path, even in large graphs. This is often referred to as the “six degrees of separation” phenomenon. [watts_collective_1998] - **Community structure:** Vertices tend to form clusters or communities, where vertices within the same community are more densely connected than those in different communities. This property is prevalent in social networks, where groups of friends or colleagues often form tightly-knit communities. [leskovec_community_2009] - **Scale-free property:** The degree distribution of the graph follows a power-law, meaning that a few vertices have a very high degree, while most vertices have a low degree. This is often observed in social networks, where a small number of individuals (e.g., celebrities) have many connections, while the majority of users have relatively few connections. [barabasi_emergence_1999]

Graph vertex embeddings are a powerful technique for representing vertices in a graph as low-dimensional vectors, enabling various machine learning tasks such as vertex classification, link prediction [leskovec_predicting_2010], and community detection. The effectiveness of these embeddings often depends on the underlying graph structure and the methods used to generate them. When faced with large graphs, the challenge of efficiently computing these embeddings while preserving the graph’s structural properties becomes paramount. Additionally, most real-world graphs are dynamic, with vertices and edges being added or removed over time. This dynamic nature introduces additional challenges in maintaining accurate and up-to-date embeddings, as the graph’s structure evolves.

Motivation to use distributed systems for graph embedding arises from the need to handle large-scale graphs that cannot fit into the memory of a single machine. Distributed systems allow for parallel processing and storage of graph data across multiple machines. However, distributing the graph across multiple machines introduces new challenges, particularly in terms of partitioning the graph and maintaining embedding quality.

When a distributed system assigns newly arriving vertices to partitions before their full neighborhood is known, it faces a practical trade-off: using more partitions improves scalability but can harm embedding quality and partition balance, while hedging an uncertain assignment by replicating a vertex across multiple partitions can recover embedding quality at the cost of additional computation. Despite its practical relevance, this trade-off has not been systematically studied in the context of dynamic graph embedding. This paper aims to fill this gap by empirically studying how partition assignment parameters, specifically the number of partitions and the replication factor, affect embedding quality, partition balance, and computational cost in distributed dynamic graph embedding systems.

Problem Formulation

Given a dynamic graph where vertices arrive online and a distributed dynamic Node2Vec-style embedding model is maintained, we want to know how partitions should be assigned to new vertices to balance:

- embedding quality,
- partition balance,
- computational cost of maintaining the assignment

Related Work

Graph vertex embedding is a well-studied area, with various methods proposed to generate low-dimensional representations of vertices in a graph. State of the art method which is widely used is Node2Vec [grover_node2vec_2016] which uses random walks to capture the local and global structure of the graph. Node2Vec generates embeddings by performing biased random walks on the graph, allowing it to explore both local and global structures. The method has been shown to be effective in capturing community structures and generating meaningful embeddings for various machine learning tasks. As its improvement, DistGER [fang_distributed_2023] is a distributed graph embedding method that extends Node2Vec by leveraging distributed computing to handle large graphs. DistGER uses a similar random walk approach but optimizes walk sampling in order to maximize the information gain when selecting the next vertex to visit.

Another approach to scale Node2Vec is proposed in [lombardo_scalable_2019] which is based on actor model and uses a distributed framework to generate embeddings for large graphs. This method allows for parallel processing of random walks, significantly improving the efficiency of embedding generation in terms of time and resource usage.

The common ground for these methods is that they generate walks which are later used to train Word2Vec model [church_word2vec_2017] commonly used for generating embeddings in natural language processing tasks.

During the learning process, there are several state of the art approaches to parallelize the training of word2vec model. Commonly used approach in distributed environment is ensemble learning [ji_ensemble_2007] which combines multiple smaller models to create a larger model. Final model is created by aggregating smaller models using parameter server architecture [li_parameter_2013].

All enumerated methods are aimed for static graphs, where the graph structure does not change over time. However, in many real-world applications, graphs are dynamic and change over time, requiring dynamic graph embedding methods that can adapt to changes in the graph structure.

The main challenge to address the problem of distributed graph vertex embedding

is partitioning the graph in a way that preserves the community structure while ensuring that the partitions are balanced and can be processed efficiently in a distributed environment. Up until recently, most partitioning methods covered only small graphs or graphs without inherent community structure, like in [benlic_effective_2010] [sanders_distributed_2012] [sanders_engineering_2011] [romero_ruiz_memetic_2018]. The main focus of these methods is static graph partitioning meaning that the graph is partitioned once and then used for processing. See [catalyurek_more_2023] for a broader, more recent survey of advances in graph and hypergraph partitioning. However, in many real-world applications, graphs are dynamic and change over time, requiring dynamic partitioning methods that can adapt to changes in the graph structure. In [ugander_balanced_2013], a variant of label propagation algorithm is proposed for balancing partitions, but it lacks the ability to adapt to changes in the graph structure over time and it does not study the impact of partitioning on embedding quality.

For dynamic graph partitioning, there are several methods available in literature like [stanton_streaming_2012] and [tsourakakis_fennel_2014]. These methods focus on partitioning dynamic graphs by considering the changes in the graph structure over time and adapting the partitioning accordingly. However, these methods have not been used in embedding applications so far, and their effectiveness in generating high-quality embeddings in a distributed environment remains an open question.

Distributed graph embedding methods, including scalable distributed approaches to static Node2Vec [lombardo_scalable_2019] and dynamic Node2Vec variants such as dynnode2vec [mahdavi_dynnode2vec_2018], enable scalable representation learning on large, evolving graphs. However, in dynamic settings with online node arrivals, a fundamental systems problem arises: newly arriving nodes must be assigned to partitions before sufficient structural or embedding information about their neighborhood is available, and it is unclear how the number of partitions and the degree of assignment redundancy affect the resulting embedding quality and partition balance.

Most existing distributed embedding systems assume either:

- static partitioning [benlic_effective_2010] [sanders_distributed_2012], where the graph is partitioned once ahead of time, or
- a batch label-propagation-style assignment such as [ugander_balanced_2013], which, although not designed for online node arrivals, illustrates the same underlying limitation: each vertex is assigned a single partition without a mechanism to hedge an uncertain assignment.

Contributions

This paper makes the following contributions:

- We propose a systematic empirical study of how partition count and replication factor affect partition assignment quality for online vertex arrivals in dynamic graph embedding systems.
- We evaluate the impact of these partition assignment parameters on embedding quality, partition balance, and edge cut, and show that increasing the replication factor can recover embedding quality lost to early, uncertain assignment at the cost of additional computation.
- We provide empirical evidence and analysis of this quality/computation trade-off, contributing to the understanding of how to effectively manage partitioning in distributed dynamic graph embedding systems.

Paper Organization

The rest of the paper is organized as follows: First, system overview is presented, then graph partitioning and embedding model are described. Next, benchmarks are explained in detail. Finally, results and discussion are presented, followed by the conclusion and references.

Methods

System overview

Temporal graph is represented as a sequence of events. Each event is a tuple (t, u, v) where t is the timestamp of the event and u and v are the vertices involved in the event. The events are processed in chronological order, and the graph is updated accordingly. The system maintains a distributed dynamic Node2Vec-style embedding model, which is updated as new events arrive. The partition assignment strategy determines how new vertices are assigned to partitions in the distributed system.

The system buffers incoming events for a short period before assigning them to partitions. This buffering allows the system to make more informed partitioning decisions by considering a batch of events together, rather than assigning partitions immediately upon the arrival of each event. Once the events are buffered, the partition assignment strategy is applied, and the embeddings are updated accordingly.

In the following listing, pseudocode for the system is presented:

```
global state buffer = {}

when new event (t, u, v) arrives:
    buffer.add((t, u, v))
    if buffer.size() >= BUFFER_SIZE:
        assign_partitions(buffer)
```

```

update_embeddings(buffer)
buffer.clear()

```

Buffer also defines new graph snapshot. Let G_n be the graph snapshot after processing the first n buffers. Now, let B_n be the n -th buffer of events. The graph snapshot G_{n+1} is obtained by applying the events in buffer B_n to the graph snapshot G_n . Specifically, $G_{n+1} = G_n \cup B_n$, where the union operation adds the new vertices and edges from the events in B_n to the existing graph snapshot G_n . This process allows the system to maintain an up-to-date representation of the dynamic graph as new events are processed.

Graph partitioning

Let $\mathcal{P} = \{p_1, \dots, p_P\}$ denote the current partitioning of the graph into P partitions. The partition of a vertex is the partition with the highest score. The partitioning strategy assigns the vertex to the partition that contains the most neighbors of the vertex in the buffer, while keeping the partition sizes balanced. The partitioner also has a replication factor, which allows it to assign a vertex to multiple partitions. The partitioner has a capacity penalty, which penalizes partitions that have more vertices than the average partition size, to encourage more balanced partitions.

Formally, the score of partition $p_i \in \mathcal{P}$ for a vertex v is defined as:

$$S(p_i, v) = N(p_i, v) - \mu \cdot \max \left(0, |p_i| - \alpha \cdot (1 + \epsilon) \cdot \frac{1}{P} \sum_{p_j \in \mathcal{P}} |p_j| \right)$$

where $N(p_i, v)$ is the number of neighbors of vertex v in partition p_i in the buffer, μ is the capacity penalty coefficient, α is the weight of the average partition size in the capacity penalty, ϵ is the imbalance tolerance, and $|p_i|$ is the size of partition p_i . The partitioner assigns the vertex to the top RF partitions of $\mathcal{P}$ with the highest scores, where RF is the replication factor. If there are multiple partitions with same scores, the partitioner randomly assigns the vertex to some of those partitions until it reaches the replication factor.

Because the partitioner only considers neighbors observed within the current buffer, its per-buffer cost is bounded independently of the total graph size processed so far. Let B denote the buffer size and let $\deg_B(v)$ denote the number of buffer-local edge endpoints incident to vertex v , so that $\sum_v \deg_B(v) \leq 2B$. For each buffered edge, looking up the current partition membership of its endpoints and incrementing the corresponding neighbor-count tallies $N(p_i, v)$ takes $O(1)$ amortized time per edge endpoint per replica, giving $O(RF \cdot B)$ total work to compute all neighbor counts for the buffer, since a vertex can belong to up to RF partitions. The average partition size term $\frac{1}{P} \sum_{p_j \in \mathcal{P}} |p_j|$ is maintained incrementally and reused across all vertices in the buffer, costing $O(P)$ per buffer rather than per vertex. For each of the at most $2B$ vertices touched by the

buffer, computing $S(p_i, v)$ for all P partitions and selecting the top RF scores costs $O(P)$ per vertex. The total cost of partitioning one buffer is therefore

$$O(B \cdot (P + RF))$$

which, for fixed P and RF , is $O(B)$: linear in the buffer size and independent of the total number of vertices or edges in the graph processed so far. Over a stream of T buffers covering $N = T \cdot B$ events in total, the cumulative partitioning cost is $O(N \cdot (P + RF))$, i.e., linear in the total number of events for the fixed, small values $P \in \{1, 2, 4, 8\}$ and $RF \in \{1, 3\}$ used in this study.

This bound is asymptotically dominated by the cost of the embedding step for the same buffer. `dynnode2vec` generates `n_walks` random walks of length `walk_size` from each of the (up to $2B$) evolving vertices, and trains skip-gram over the resulting corpus for several epochs, at a cost of $O(|\Delta V_t| \cdot n_walks \cdot walk_size \cdot window_size \cdot epochs)$ per partition replica. Both costs are linear in the number of vertices touched by the buffer, but the constant factor for embedding ($walks \times walk\ length \times window\ size \times epochs$) is much larger than partitioning’s constant factor ($P + RF \leq 11$ in this study), which is why partitioning time is negligible compared to embedding time in practice.

Embedding model

Given dynamic graph (i.e., a graph that changes over time), the `dynnode2vec` algorithm [mahdavi_dynnode2vec_2018] learns continuous feature representations for nodes in the graph at different time steps. The main idea behind `dynnode2vec` is to extend the `node2vec` algorithm [grover_node2vec_2016] to handle dynamic graphs by incorporating temporal information into the random walk strategy and embedding learning process. The goal is to capture both the structural and temporal dynamics of the graph in the learned embeddings.

In the following listing, pseudocode for the `dynnode2vec` algorithm is presented:

Input: Graph snapshots $G_1, G_2, \dots, G_T$

Output: Embeddings $Z_1, Z_2, \dots, Z_T$

```
// t = 1
Run static node2vec on G1 to train Skip-gram_1 and obtain Z1.

For t = 2 ... T:
    // Identify evolving nodes between G_{t-1} and G_t
    Compute:
        V_add = nodes added from t-1 to t
        E_add = edges added from t-1 to t
        E_del = edges deleted from t-1 to t
    delta_V_t = V_add union { v in V_t | exists (v, u) in (E_add union E_del) }
```

```

// Evolving random walks only from changed regions
Walk_n = node2vec_walks(G_t, start_nodes=delta_V_t, p, q, walk_length, num_walks)

// Dynamic Skip-gram: initialize from previous time
Initialize Skip-gram_t with weights of Skip-gram_{t-1}
Update Skip-gram_t vocabulary for any new vertices
Train Skip-gram_t on Walk_n
Extract Z_t from Skip-gram_t
End For

```

In this paper, graph is represented as a sequence of events. Algorithm can be easily adapted to this representation by buffering events and applying them to the graph snapshot at each time step. The algorithm captures the evolving nature of the graph by focusing on the changed regions and updating the embeddings accordingly.

Since one vertex can be assigned to multiple partitions, each partition produces one embedding for corresponding vertex. Let $z_p = h_p(u)$ be the embedding of vertex u in partition p . The final embedding of vertex u is obtained by averaging the embeddings from all partitions that contain the vertex:

$$z_u = \frac{1}{|P_u|} \sum_{p \in P_u} z_p$$

where $P_u = \{p | u \in V_p\}$ is the set of partitions that contain vertex u . This averaging process allows the system to combine information from multiple partitions, potentially improving the quality of the final embedding by leveraging diverse perspectives on the vertex's relationships within the graph. This step requires communication between partitions to share the embeddings of vertices that are replicated across multiple partitions. The communication overhead is proportional to the number of replicated vertices and the size of their embeddings.

If replication factor is set to 1, then the final embedding of vertex u is simply the embedding from the single partition that contains the vertex:

$$z_u = z_p$$

where p is the partition that contains vertex u . In this case, the embedding is solely based on the information available in that single partition, which may limit the quality of the embedding if the partition does not capture sufficient context about the vertex's relationships within the graph.

Additionally, if vertex is not partitioned yet, then the final embedding of vertex u is set to zero vector:

$$z_u = 0$$

Benchmarks and evaluation metrics

To evaluate the effectiveness of the graph vertex embeddings generated using community-aware partitioning, several criteria are considered:

Embedding Quality: The quality of the embeddings is assessed using F1-score of reconstructed graph [yip_restore_2023], which measures how well the embeddings capture the relationships between vertices in the original graph. Higher F1-scores indicate better preservation of graph structure in the embeddings. Let $G = (V, E)$ be the original graph and $G' = (V, E')$ be the reconstructed graph from embeddings. G' is constructed by connecting the m closest vertex pairs in the embedding space, where m is the number of edges in the original graph. The F1-score is calculated as follows:

$$F1 = 2 * \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

where

$$\text{Precision} = \frac{1}{|V|} \sum_{v \in V} \frac{|N(v) \cap N'(v)|}{|N'(v)|}$$

$$\text{Recall} = \frac{1}{|V|} \sum_{v \in V} \frac{|N(v) \cap N'(v)|}{|N(v)|}$$

and $N(v)$ and $N'(v)$ are the neighbors of vertex v in the original and reconstructed graphs, respectively.

Partition Quality: The quality of the partitions is evaluated using edge cut, which measures the number of edges that connect vertices in different partitions. Lower edge cuts indicate better preservation of community structures within partitions. Edge cut is defined as:

$$\text{EdgeCut} = \frac{1}{|E|} \sum_{(u,v) \in E} \mathbb{I}(p(u) \neq p(v))$$

where E is the set of edges in the original graph, $p(u)$ is the partition of vertex u , and $\mathbb{I}(\cdot)$ is the indicator function.

Balance of Partitions: The balance of the partitions is assessed by measuring the size of each partition and ensuring that they are within a bounded interval. This helps to ensure that the workload is evenly distributed across machines in the distributed environment. Formally, balance is defined as:

$$B = 1 + \frac{1}{K} \sum_{i=1}^K \frac{|s_i - \bar{s}|}{\bar{s}}$$

where K is the number of partitions, s_i is the number of vertices assigned to partition i , and $\bar{s} = \frac{1}{K} \sum_{i=1}^K s_i$ is the average partition size. A perfectly balanced partitioning yields $B = 1$; higher values indicate greater imbalance.

Partitioning Time: The time taken to partition the graph is measured to evaluate the efficiency of the partitioning algorithms. Faster partitioning times are preferred, especially for large graphs, as they reduce the overall processing time in a distributed environment. In this paper, partitioning time is analyzed asymptotically via the per-buffer complexity bound derived in the Graph partitioning section, and validated qualitatively against the embedding time observed in the experiments, rather than measured as standalone wall-clock numbers.

Repartitioning amount: The amount of repartitioning required when new vertices arrive is measured to assess the stability of the partitioning strategy. Lower amounts of repartitioning indicate that the partitioning strategy is more stable and can handle dynamic changes in the graph without significant disruption. Given two graph snapshots G_n and G_{n+1} , the amount of repartitioning is defined as:

$$\text{RepartitioningAmount} = \frac{1}{|V_n|} \sum_{v \in V_n} \mathbb{I}(p_n(v) \neq p_{n+1}(v))$$

Where V_n is the set of vertices in graph snapshot G_n , $p_n(v)$ is the partition of vertex v in snapshot G_n , and $p_{n+1}(v)$ is the partition of vertex v in snapshot G_{n+1} . This metric quantifies the proportion of vertices that have been reassigned to different partitions between two consecutive snapshots, providing insight into the stability and adaptability of the partitioning strategy in response to dynamic changes in the graph. It has to be noted that only vertices that are present in both snapshots are considered for this metric, as newly added vertices do not have a previous partition assignment to compare against and thus do not contribute to the measure of repartitioning amount.

Results and discussion

Partitioning hyperparameters

Throughout the experiments, the following hyperparameters are used for partitioning:

Table 1: Partitioning hyperparameters used throughout the experiments.

Hyperparameter	Value
Buffer size	1000
μ	1

Hyperparameter	Value
α	1
ϵ	0.1
Partitions (P)	$\{1, 2, 4, 8\}$
Replication factor (RF)	$\{1, 3\}$

These hyperparameters are chosen to balance the trade-offs between embedding quality, partition balance, and computational cost. In particular, the buffer size is set to 1000 to allow for sufficient information to be gathered before making partitioning decisions; this value is held fixed throughout the experiments, and its effect is not directly evaluated in this study. The capacity penalty coefficient (μ) is set to 1 to encourage balanced partitions, while the weight of the average partition size (α) is also set to 1 to ensure that the capacity penalty is proportional to the average partition size. The imbalance tolerance (ϵ) is set to 0.1 to allow for some flexibility in partition sizes while still encouraging balance. The number of partitions (P) is varied between 1, 2, 4, and 8 to evaluate the impact of partitioning strategy on embedding quality and partition balance. Finally, the replication factor (RF) is varied between 1 and 3 to analyze the effect of replicating vertices across multiple partitions on embedding quality.

For the embedding model itself, the following dataset-specific hyperparameters are used, obtained from a preliminary search over the node2vec return/in-out parameters p, q and the embedding dimension:

Table 2: Embedding hyperparameters (node2vec p, q , and embedding dimension) used for each dataset.

Dataset	p	q	dim
CITESEER	0.5	0.25	50
DBLP	0.5	1	25
AstroPh	2	0.25	50
AS-Oregon	0.5	2	128
Enron	0.5	1	128

Datasets

Experiments are performed on five real-world graph datasets, summarized in the table below.

Table 3: Datasets used in the experiments.

Dataset	Nodes	Edges	Average degree	Average cluster-ing coefficient	Density	Modularity
CITeseer	3264	4536	2.78	0.145	0.0009	0.72
DBLP	17716	52867	5.97	0.134	0.00034	0.58
AstroPh	18772	198110	21.11	0.631	0.00112	0.33
AS-Oregon	11806	38781	6.57	0.399	0.00056	0.57
Enron	36692	183831	10.02	0.497	0.00027	0.36

CITeseer is a citation network of scientific publications. DBLP and AstroPh are collaboration/citation networks derived from SNAP, where AstroPh links co-authors of astrophysics papers. AS-Oregon is a network of autonomous systems and their peering connections. Enron is an email communication network in which vertices are email addresses and edges represent messages exchanged between them. These datasets vary in size and density, which allows the effect of partitioning to be observed under different community structures.

As a static, non-distributed reference point, running node2vec on the full, unpartitioned graph gives the following F1 reconstruction scores:

Table 4: Average F1 reconstruction score of static node2vec on the full, unpartitioned graph for each dataset.

Dataset	Average F1 score
CITeseer	34.82%
DBLP	60.3%
AstroPh	70.41%
AS-Oregon	31.26%
Enron	20.42%

These values serve as a rough upper bound against which the effect of partitioning, buffering, and dynamic re-embedding can be compared: a system that partitions the graph and processes it incrementally is generally not expected to exceed this quality, since it operates with a fraction of the graph’s global information at any given time and adapts to it incrementally rather than embedding the full graph at once. CITeseer is a notable exception to this expectation, discussed in the next section: the buffered dynnode2vec pipeline exceeds the static baseline there even without partitioning, plausibly because repeated incremental re-embedding

across buffers gives the model more effective training signal than a single static pass over the graph.

Sensitivity analysis of the balance penalty parameter

The most common neighbor partitioner uses a capacity penalty coefficient μ to discourage assigning vertices to partitions that are already larger than average. This section analyzes the effect of μ on partition balance, varying $\mu \in \{0, 0.5, 1, 1.5, 2\}$ across all five datasets with 2 and 4 partitions. $\mu = 0$ disables the penalty entirely, so balance is determined purely by neighbor counts; higher μ increasingly rebalances vertex placement at the expense of prioritizing neighbor locality.

The tables below show the average balance at the final iteration for each combination of μ and partition count. $B = 1$ is a perfectly balanced partitioning; higher values indicate greater imbalance.

Table 5: Average balance at the final iteration for AS-Oregon dataset across different μ values and partition counts.

Penalty	P=2	P=4
0.0	2.00	2.50
0.5	1.18	1.49
1.0	1.18	1.20
1.5	1.10	1.21
2.0	1.05	1.19

Table 6: Average balance at the final iteration for AstroPh dataset across different μ values and partition counts.

Penalty	P=2	P=4
0.0	2.00	2.50
0.5	1.15	1.18
1.0	1.13	1.12
1.5	1.08	1.12
2.0	1.05	1.22

Table 7: Average balance at the final iteration for CITESEER dataset across different μ values and partition counts.

Penalty	P=2	P=4
0.0	2.00	2.50
0.5	1.61	1.61

Penalty	P=2	P=4
1.0	1.31	1.63
1.5	1.17	1.50
2.0	1.17	1.17

Table 8: Average balance at the final iteration for DBLP dataset across different μ values and partition counts.

Penalty	P=2	P=4
0.0	2.00	2.50
0.5	1.08	1.16
1.0	1.03	1.09
1.5	1.14	1.20
2.0	1.01	1.14

Table 9: Average balance at the final iteration for Enron dataset across different μ values and partition counts.

Penalty	P=2	P=4
0.0	2.00	2.50
0.5	1.10	1.14
1.0	1.10	1.10
1.5	1.02	1.02
2.0	1.06	1.06

The tables below show the average balance across all iterations (not just the last one) for each combination of μ and partition count, capturing how balance behaves during the stream rather than only at the end.

Table 10: Average balance across all iterations for AS-Oregon dataset across different μ values and partition counts.

Penalty	P=2	P=4
0.0	2.00	2.50
0.5	1.14	1.51
1.0	1.12	1.23
1.5	1.10	1.18
2.0	1.10	1.15

Table 11: Average balance across all iterations for AstroPh dataset across different μ values and partition counts.

Penalty	P=2	P=4
0.0	2.00	2.50
0.5	1.24	1.51
1.0	1.14	1.27
1.5	1.11	1.21
2.0	1.10	1.20

Table 12: Average balance across all iterations for CITESEER dataset across different μ values and partition counts.

Penalty	P=2	P=4
0.0	2.00	2.50
0.5	1.92	2.12
1.0	1.52	1.84
1.5	1.34	1.66
2.0	1.34	1.55

Table 13: Average balance across all iterations for DBLP dataset across different μ values and partition counts.

Penalty	P=2	P=4
0.0	2.00	2.50
0.5	1.30	1.60
1.0	1.17	1.40
1.5	1.15	1.33
2.0	1.12	1.25

Table 14: Average balance across all iterations for Enron dataset across different μ values and partition counts.

Penalty	P=2	P=4
0.0	2.00	2.50
0.5	1.14	1.27
1.0	1.11	1.19
1.5	1.09	1.17
2.0	1.08	1.15

The following plots show the (smoothed) average balance over iterations for each μ value, for $P=2$ and $P=4$ partitions side by side.

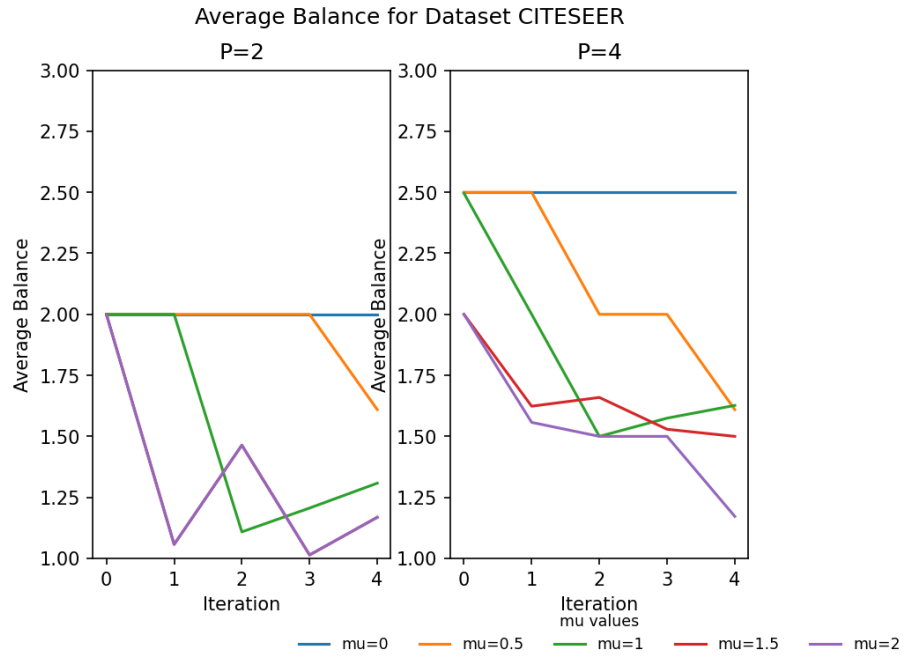
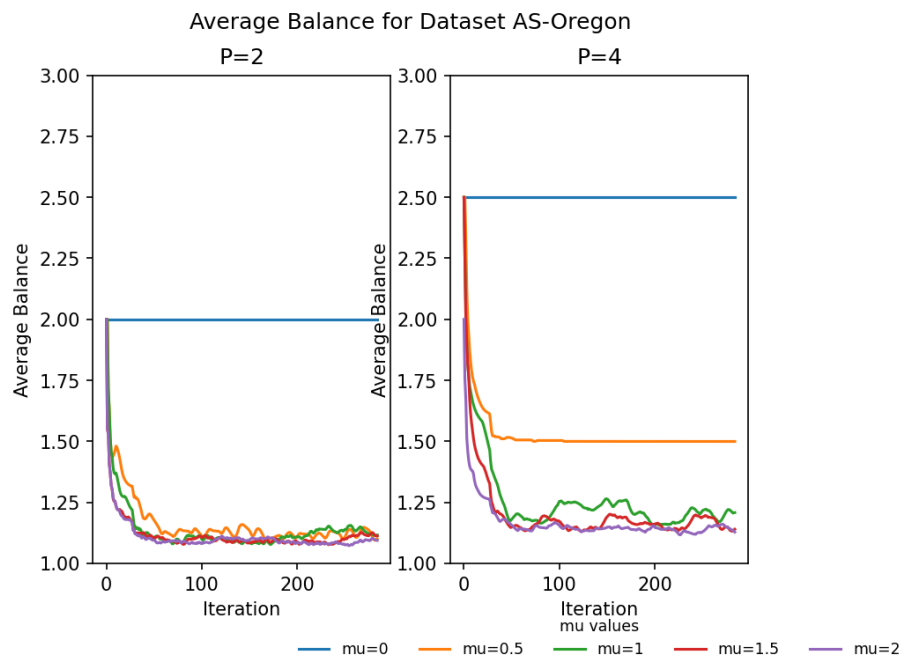
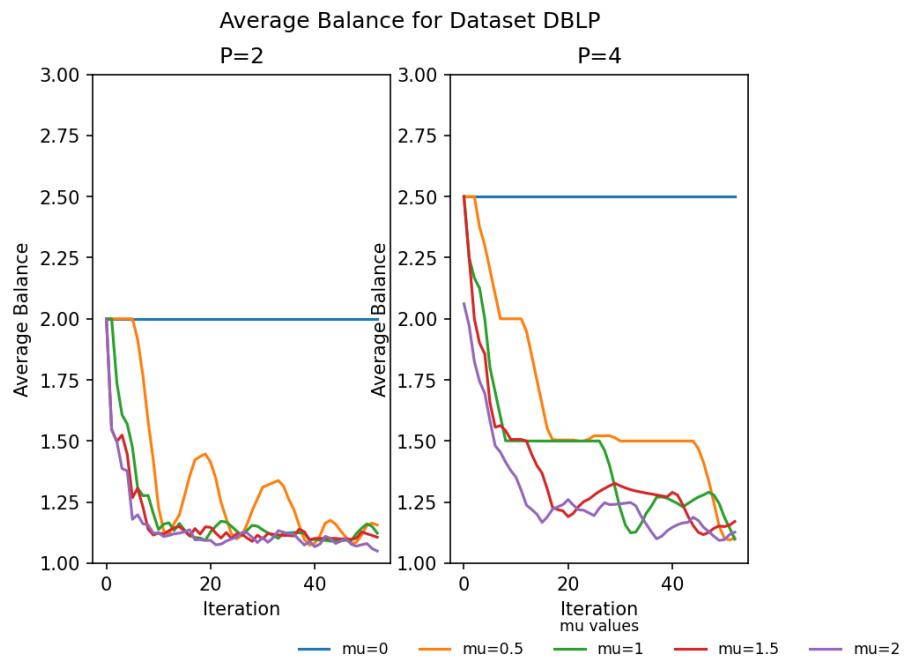


Figure 1: CITESEER Balance vs μ



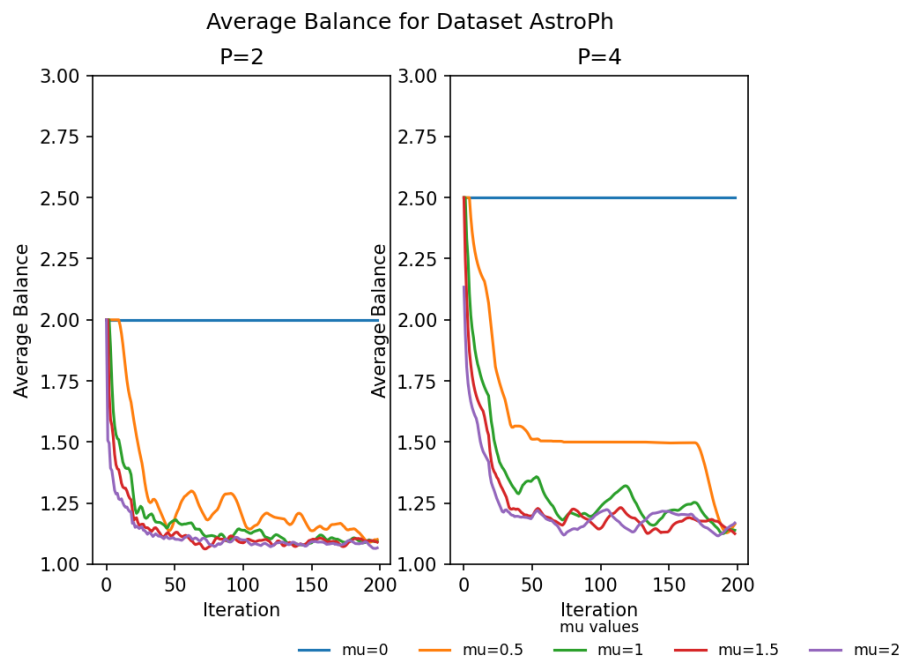
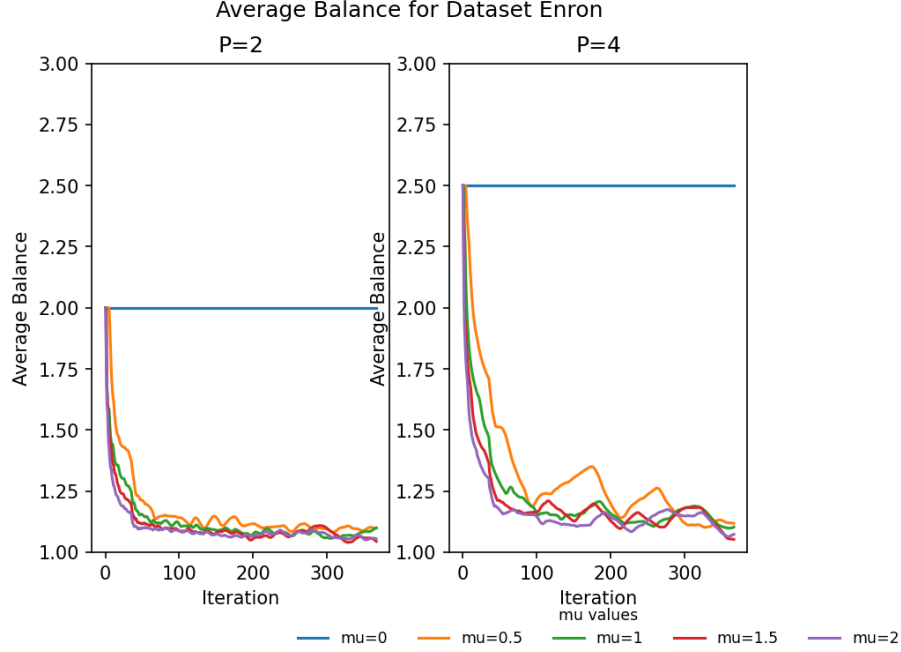


Figure 2: AstroPh Balance vs mu



Across all datasets, $\mu = 0$ (no capacity penalty) produces the worst balance, matching the partition count almost exactly ($B \approx 1.5 + P/4$ at P partitions), since vertices are placed purely by neighbor affinity with no regard for partition size. Increasing μ from 0 to 1 yields the largest balance improvement; beyond $\mu = 1$ – 1.5 , returns diminish and balance mostly plateaus, with CITESEER remaining the hardest dataset to balance at every μ value tested.

Buffer size sensitivity analysis

We analyzed the impact of different buffer sizes on the embedding quality and time performance of the buffered dynnode2vec algorithm.

The plots above show the F1 reconstruction score and the time taken for embedding as a function of buffer size for CITESEER and AstroPh datasets. The results indicate that larger buffer sizes generally lead to better embedding quality, as more information is available for partitioning and embedding. However, larger buffers also increase the time taken for partitioning, as more vertices and edges need to be processed in each buffer.

Embedding quality, partition balance and edge cut

Using the temporal test command (`vv temporal_test`) with the buffered event processing pipeline described in the System overview, the following tables report,

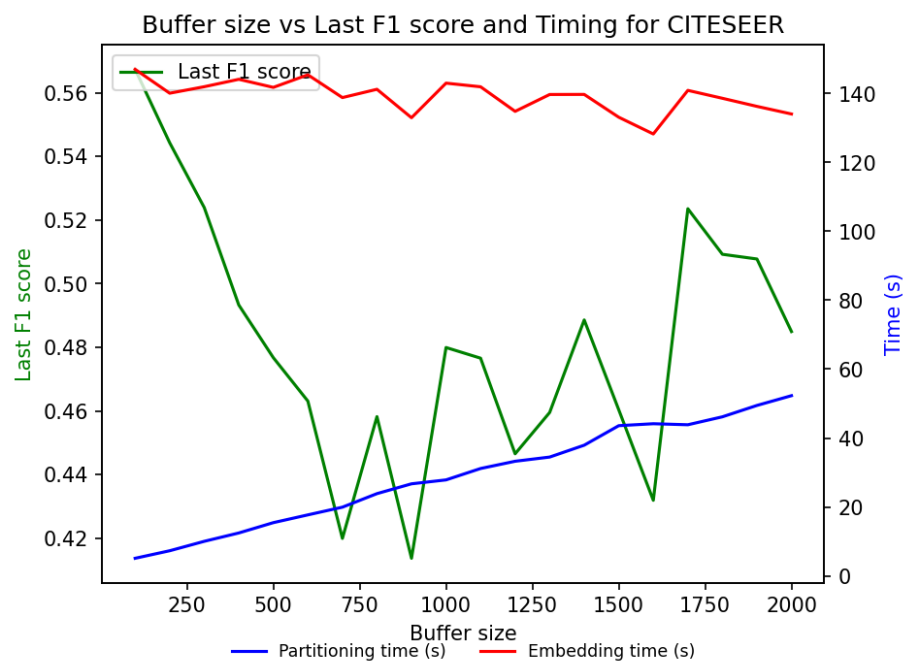


Figure 3: Buffer size sensitivity analysis

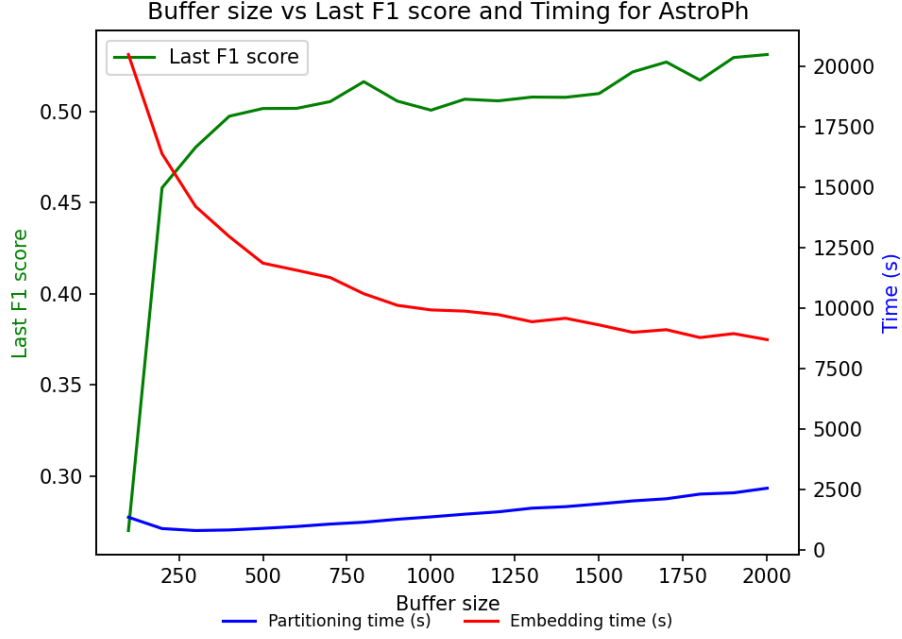


Figure 4: Buffer size sensitivity analysis

for each dataset and number of partitions $P \in \{1, 2, 4, 8\}$ with replication factor $RF = 1$, the F1 reconstruction score, the edge cut, and the balance of the resulting partitions, averaged over 10 iterations of the buffered stream.

Table 15: F1 reconstruction score, edge cut, and balance for CITE-SEER with buffered dynnode2vec, $RF = 1$.

P	F1	Edge cut	Balance
1	47.57% $\pm$ 1.56%	0%	1
2	45.81% $\pm$ 0.59%	23%	1.31
4	48.06% $\pm$ 0.53%	39%	1.63
8	54.81% $\pm$ 1.66%	48%	1.75

Table 16: F1 reconstruction score, edge cut, and balance for DBLP with buffered dynnode2vec, $RF = 1$.

P	F1	Edge cut	Balance
1	50.91% $\pm$ 0.31%	0%	1
2	54.11% $\pm$ 0.18%	32%	1.10

P	F1	Edge cut	Balance
4	54.51% $\pm$ 0.15%	50%	1.20
8	49.87% $\pm$ 0.16%	59%	1.31

Table 17: F1 reconstruction score, edge cut, and balance for AstroPh with buffered dynnode2vec, RF = 1.

P	F1	Edge cut	Balance
1	57.70% $\pm$ 0.11%	0%	1
2	54.67% $\pm$ 0.36%	35%	1.08
4	50.95% $\pm$ 0.45%	56%	1.11
8	45.94% $\pm$ 0.51%	65%	1.26

Table 18: F1 reconstruction score, edge cut, and balance for AS-Oregon with buffered dynnode2vec, RF = 1.

P	F1	Edge cut	Balance
1	26.17% $\pm$ 0.57%	0%	1
2	24.88% $\pm$ 2.72%	30.44%	1.21
4	24.30% $\pm$ 0.98%	53.79%	1.26
8	21.45% $\pm$ 1.19%	71.56%	1.27

Table 19: F1 reconstruction score, edge cut, and balance for Enron with buffered dynnode2vec, RF = 1.

P	F1	Edge cut	Balance
1	18.03% $\pm$ 0.44%	0%	1
2	24.84% $\pm$ 0.38%	33.07%	1.09
4	27.44% $\pm$ 2.83%	49.18%	1.08
8	32.36% $\pm$ 1.24%	61.99%	1.25

As expected, edge cut increases monotonically with the number of partitions across all datasets, since splitting the vertex set into more partitions necessarily severs more cross-community edges. Balance stays close to the ideal value of 1 for DBLP, AstroPh, AS-Oregon, and Enron (at most 1.35 even at $P = 8$), confirming that the neighbor-based partitioner with capacity penalty distributes vertices close to evenly on these datasets without an explicit load-balancing step. CITESEER is a notable exception: its balance grows to 1.63 at $P = 4$ and 1.75 at $P = 8$, indicating a substantially less even partition size distribution than on the other datasets under the same capacity penalty settings.

The effect of partitioning on embedding quality (F1), however, is dataset-dependent. On CITESEER, DBLP, and Enron, F1 score remains stable or even improves as P increases, despite the growing edge cut: this suggests that, on these graphs, most of a vertex’s relevant neighborhood is captured within its own partition, so splitting the graph does not meaningfully harm the quality of the embedding that is reconstructed from local, per-partition context. CITESEER stands out further still: even at $P = 1$, its buffered F1 score (47%) already exceeds the static node2vec baseline (34.82%) reported above, and it continues to rise with P , reaching 49.86% at $P = 8$. On AstroPh and AS-Oregon, F1 is lower at $P = 8$ than at $P = 1$, since a larger share of each vertex’s neighbors end up outside its assigned partition, and this information is not available to the local embedding process. AstroPh degrades monotonically with P , while AS-Oregon first rises slightly at $P = 2$ (26.58% to 28.72%) before falling at $P = 4$ and $P = 8$, so the downward trend is not strictly monotonic for every dataset in this group.

Across the five datasets, the magnitude of the F1 score change from $P = 1$ to $P = 8$ correlates most strongly with graph density (Spearman $\rho \approx -0.7$), and more weakly with modularity ($\rho \approx +0.4$) and average clustering coefficient ($\rho \approx -0.3$). However, with only five datasets, none of these statistics cleanly separates the two groups above with a single threshold: CITESEER, part of the stable/improving group, is denser than AS-Oregon, which degrades; and Enron combines the lowest density, the lowest modularity among the stable group, and only moderate clustering, yet shows the largest F1 improvement of any dataset in the study. This suggests density is the closest available proxy among the properties measured here, but not a complete explanation, and a larger dataset sample would be needed to identify the actual driver with confidence.

Partitioning time itself was found to be negligible compared to embedding time in all experiments, consistent with the $O(B \cdot (P + RF))$ per-buffer complexity bound derived in the Graph partitioning section, whose constant factor is much smaller than the embedding step’s; the dominant cost of processing a buffer is the embedding computation on the slowest (largest or most active) partition.

Effect of replication factor

Increasing the replication factor RF allows a vertex to be embedded independently in more than one partition, with the final embedding obtained by averaging the per-partition embeddings as described in the Embedding model section. This trades additional computation and storage for a more informed, less committal partition assignment. In a smaller exploratory hyperparameter sweep on CITESEER with $P = 4$, buffer size 1000, and capacity penalty $\mu = 1$ (run separately from, and not directly comparable to, the main results table above, which used a longer run), increasing the replication factor from $RF = 1$ to $RF = 3$ raised the F1 reconstruction score from 44.35% to 69%, indicating that hedging the partition assignment across multiple partitions can substantially mitigate the quality loss introduced by early, uncertain assignment decisions. This comes at

the cost of up to $RF \times$ the embedding computation per vertex, illustrating a quality/computation trade-off: assigning a vertex to a single partition is cheaper but riskier, while replicating it across several partitions is more expensive but more robust to a suboptimal initial assignment. Because this observation comes from a single, small-scale exploratory run, it should be read as preliminary evidence rather than a robust result; a systematic replication factor sweep across datasets and partition counts is left to future work.

Temporal evolution

Beyond aggregate scores, it is informative to track how F1 score, balance, edge cut, and repartitioning rate evolve as the event stream is ingested buffer by buffer, broken down by number of partitions, for each of the five datasets.

F1 score over iterations

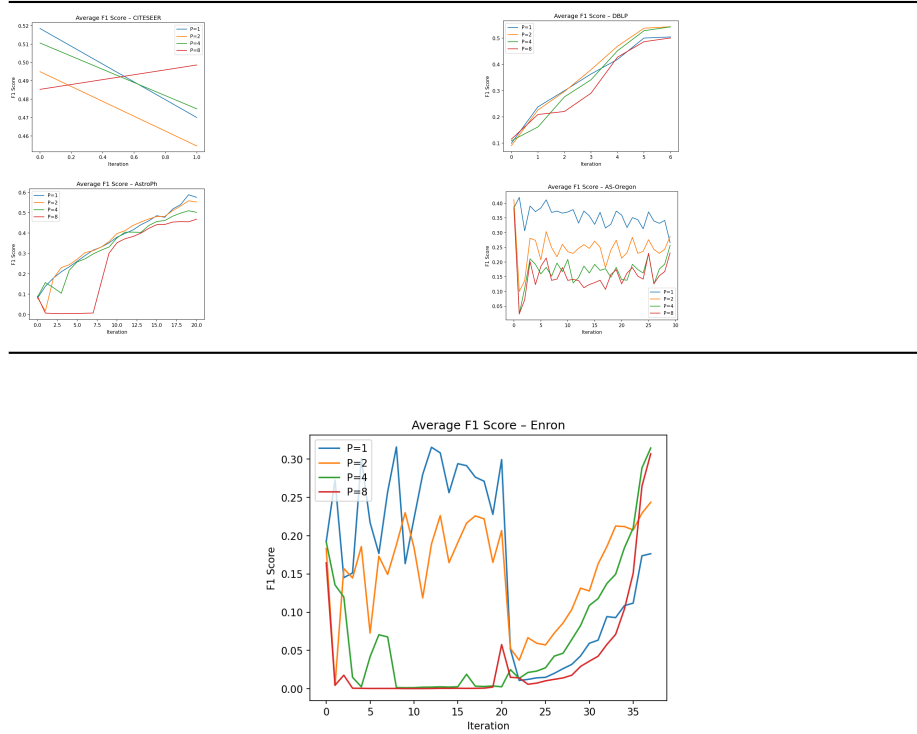


Figure 5: Enron

The F1 score generally improves as more events are ingested and the graph snapshot grows, since the embedding model accumulates more structural information over time. Using more partitions tends to remain competitive with, or even improve on, the single-partition baseline on datasets with clear community

structure (CITESEER, DBLP, Enron), while the denser AstroPh and AS-Oregon datasets show a modest drop at higher partition counts, consistent with the increased edge cut discussed above.

Balance over iterations

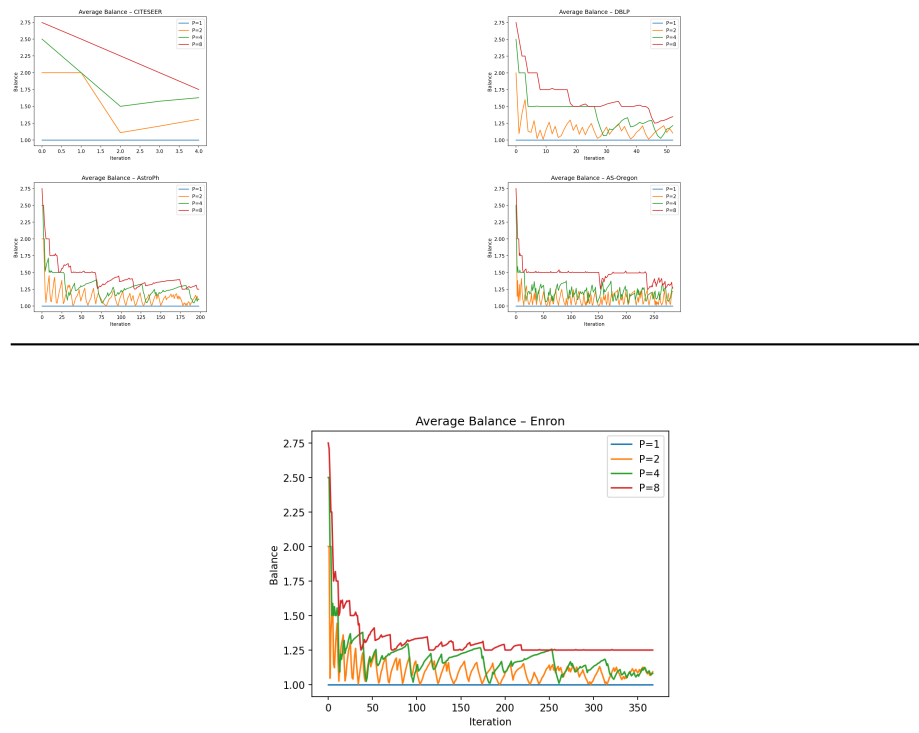


Figure 6: Enron

Balance remains close to 1 throughout the stream for DBLP, AstroPh, AS-Oregon, and Enron, showing that the neighbor-based partitioner keeps partitions nearly evenly sized on these datasets without requiring an explicit rebalancing step, even as the graph evolves. As in the aggregate results, CITESEER is the exception, settling at a noticeably higher balance value at larger partition counts.

Edge cut over iterations

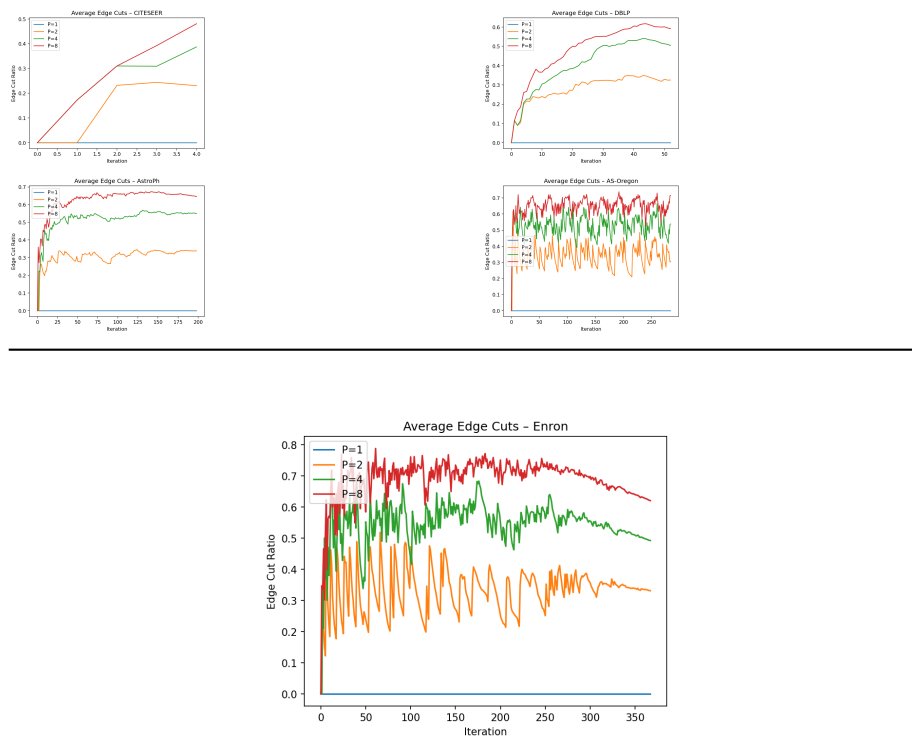
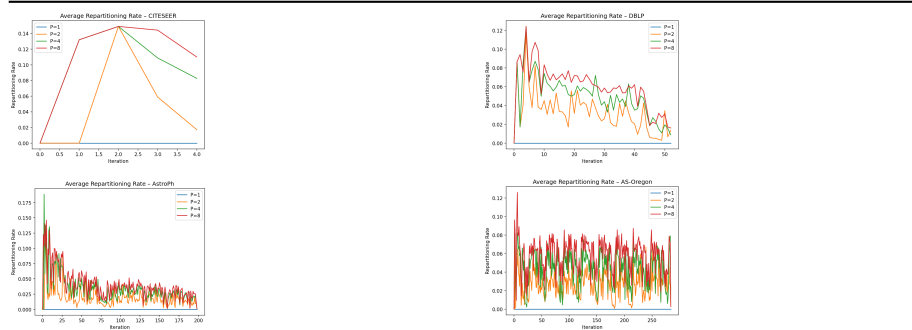


Figure 7: Enron

The edge cut ratio rises quickly during the first few buffers, while the partitions are still forming, and then stabilizes at a plateau largely determined by the number of partitions and the graph's community structure. This holds across all five datasets, indicating that the partitioner reaches a steady state rather than degrading further as the stream continues.

Repartitioning rate over iterations



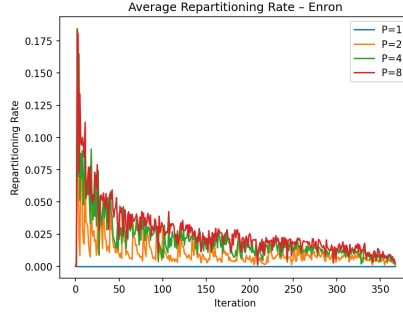


Figure 8: Enron

The fraction of vertices repartitioned per buffer decays rapidly after an initial warm-up phase and stays below 20% across all datasets and partition configurations. This indicates that, once the partitioner has seen enough of a vertex’s neighborhood, its assignment stabilizes and is only revised when the vertex’s local structure changes significantly, keeping the overhead of maintaining partition assignments low throughout the stream.

Conclusion

In this paper, we studied the trade-off between early and informed partition assignment in distributed dynamic graph embedding, in a setting where a distributed dynnode2vec-style embedding model processes graph events in buffered batches. We described a neighbor-based partitioning strategy with a tunable replication factor and capacity penalty, and used it to empirically characterize how partition count and replication factor affect embedding quality, partition balance, edge cut, and repartitioning stability across five real-world dynamic graph datasets.

The results show that the impact of partitioning on embedding quality is strongly dataset-dependent: on CITESEER, DBLP, and Enron, increasing the number of partitions has little to no negative effect on the F1 reconstruction score, despite the corresponding increase in edge cut, whereas on AstroPh and AS-Oregon embedding quality degrades noticeably as the number of partitions grows. Of the dataset properties measured (density, clustering coefficient, modularity), graph density correlates most closely with this split, though it does not fully explain it, since no single threshold on density, clustering coefficient, or modularity cleanly separates the two groups; identifying the underlying driver is left to future work. On most datasets, the partitioner keeps balance close to the ideal value, though CITESEER shows a more pronounced imbalance at higher partition counts; across all datasets and partition counts it keeps the repartitioning rate low and

decaying after an initial warm-up period, indicating that a simple neighbor-based, buffer-driven partitioning strategy is a practical and largely, though not universally, stable choice for online graph partitioning. Partitioning time itself is negligible relative to embedding time, confirming that the dominant systems bottleneck in this setting is the embedding computation on the largest or most active partition rather than the assignment decision itself.

The replication factor experiments provide direct evidence for the trade-off that motivates this paper: hedging a vertex’s partition assignment across multiple partitions and averaging the resulting embeddings can substantially mitigate the quality loss caused by early, uncertain assignment decisions, at the cost of additional computation proportional to the replication factor. This suggests that systems which must assign vertices to partitions immediately, with little information about their eventual neighborhood, benefit from replication as a way to hedge against a suboptimal initial assignment, while systems that can tolerate additional latency may instead delay assignment until more information is available, avoiding the replication overhead altogether. Together, these findings indicate that neither immediate nor delayed assignment is uniformly preferable; the right choice depends on the graph’s community structure and on which resource, latency or computation, is more constrained in a given deployment.

As future work, a more direct comparison between delayed and immediate assignment strategies is warranted, explicitly measuring assignment latency alongside embedding quality, rather than using the replication factor solely as a proxy for the trade-off. Extending the replication factor study beyond a single dataset and partition count would also strengthen the empirical evidence presented here. Finally, evaluating the resulting embeddings on downstream tasks such as link prediction and node classification, in addition to the reconstruction F1 score used in this paper, would help establish whether the observed trends generalize beyond graph reconstruction.

References